Winslow — Architectural Overview

1. System Overview

Winslow is a project management suite built as a monorepo with two main components: an F# REST API (backend) and a Flutter cross-platform client (frontend). The architecture follows **Domain-Driven Design (DDD)** with **Railway-Oriented Programming** for error handling, **CQRS** for separating reads and writes, and a **plugin-based frontend** for feature isolation.

```
graph TD
    subgraph "Frontend (Flutter / Dart)"
        App["Winslow App - main.dart"]
        PS["Plugin System"]
        RR["Requirements Feature"]
        ApiClient["ApiClient - Dio Wrapper"]
    end

    subgraph "Backend (F# / .NET 10)"
        API["Winslow.API - Falco 4.x"]
        APP["Winslow.Application"]
        DOM["Winslow.Domain"]
        INFRA["Winslow.Infrastructure"]
    end

    subgraph "Data Layer"
        PG[("PostgreSQL 16")]
        IM[("InMemory Store - dev mode")]
    end

    App --> ApiClient
    ApiClient -->|HTTP JSON| API
    API --> APP
    APP --> DOM
    APP --> INFRA
    INFRA --> IM
    INFRA -.->|via DSN| PG
```

Layer Dependency Graph (Backend)

```
graph LR
    DOM["Winslow.Domain - pure logic, no deps"]
    APP["Winslow.Application - ports, commands, handlers"]
    INFRA["Winslow.Infrastructure - repos, publishers"]
    API["Winslow.API - Falco routes, DTOs"]

    APP -->|references| DOM
```

```
    INFRA -->|references| DOM
    INFRA -->|references| APP
    API -->|references| DOM
    API -->|references| APP
    API -->|references| INFRA
```

---

2. Backend Architecture

2.1 Layer Breakdown

```
graph TB
    subgraph "Winslow.API - Falco HTTP Layer"
        Routes["GET/POST/PATCH endpoints"]
        DTOs["CreateRequirementDto / TransitionStatusDto"]
        ErrorHandler["handleError - 404/400/500"]
        Parsers["parsePriority / parseKind / parseStatus"]
    end

    subgraph "Winslow.Application - Use Cases"
        CmdHandler["CommandHandlers"]
        QueryHandler["QueryHandlers"]
        Ports["IRequirementRepository / IEventPublisher"]
        TaskResultCE["taskResult builder"]
    end

    subgraph "Winslow.Domain - Pure Domain Logic"
        ReqAgg["Requirement Aggregate Root"]
        StatusSM["Status State Machine"]
        Events["RequirementEvent"]
        Errors["DomainError"]
        Types["Common Types"]
        ResultCE["result builder"]
    end

    subgraph "Winslow.Infrastructure - Adapters"
        InMemRepo["InMemoryRequirementRepository"]
        InMemPub["InMemoryEventPublisher"]
    end

    Routes --> DTOs
    DTOs --> CmdHandler
    DTOs --> QueryHandler
    CmdHandler --> Ports
    QueryHandler --> Ports
    CmdHandler --> TaskResultCE
```

```
CmdHandler -->|create / transitionStatus| ReqAgg
ReqAgg --> StatusSM
ReqAgg --> Events
ReqAgg --> Errors
ReqAgg --> Types
ReqAgg --> ResultCE
Ports --> InMemRepo
Ports --> InMemPub
ErrorHandler --> Errors
```

2.2 Domain Layer (`Winslow.Domain`)

The innermost layer with **zero dependencies**. It contains:

Module	Contents
`Common/Types.fs`	`RequirementId`, `ProjectId`, `UserId`, `Timestamp`, `NonEmptyString`, `ResultBuilder`
`Common/Errors.fs`	`DomainError` (5 cases), `AppError` (3 cases)
`Requirements/RequirementTypes.fs`	`RequirementStatus`, `RequirementPriority` (MoSCoW), `RequirementKind`, `AcceptanceCriteria`
`Requirements/RequirementEvents.fs`	`RequirementEvent` union with 4 event types
`Requirements/Requirement.fs`	`Requirement` record, `create`, `update`, `transitionStatus`
`Projects/ProjectTypes.fs`	`ProjectStatus`, `ProjectMethodology`
`Projects/Project.fs`	`Project` record, `create`

All domain functions are **pure** — they take input, validate, and return `Result<value, DomainError>` with no side effects.

2.3 Application Layer (`Winslow.Application`)

Orchestrates use cases via **Command Handlers** and **Query Handlers**. Depends only on the Domain layer.

Ports (interfaces): - `IRequirementRepository` — `FindById`, `FindByProject`, `Save`, `Delete` - `IEventPublisher` — `Publish`

Handlers use the `taskResult { }` computation expression to chain async operations with Railway-Oriented Programming:

```
Command -> Domain function -> repo.Save -> publisher.Publish -> Result
```

Read Models are string-typed DTOs (all DU wrappers unwrapped) for direct JSON serialization.

2.4 Infrastructure Layer (`Winslow.Infrastructure`)

Implements the ports:

Adapter	Implementation	Notes
`InMemoryRequirementRepository`	`ConcurrentDictionary<Guid, Requirement>`	Seeds 2 demo requirements, no persistence
`InMemoryEventPublisher`	`printfn`	Logs event type + description to console

2.5 API Layer (`Winslow.API`)

Single `Program.fs` using **Falco 4.x** with:

- `System.Text.Json` (camelCase, case-insensitive)
- Manual string-to-DU parsing via helper functions
- Centralized `handleError` mapping `AppError` -> HTTP status codes

Endpoint	Handler	Returns
`GET /projects/{projectId}/requirements`	`apiGetProjectRequirements`	`RequirementListItem[]`
`GET /requirements/{id}`	`apiGetRequirementById`	`RequirementReadModel`
`POST /requirements`	`apiCreateRequirement`	`201 { id: "guid" }`
`PATCH /requirements/{id}/status`	`apiTransitionStatus`	`200 { status: "ok" }`

2.6 Data Flow Example — Create Requirement

```
sequenceDiagram
    participant Client as Flutter
    participant API as Winslow.API
    participant App as Application
    participant Domain as Domain
    participant Infra as Infrastructure

    Client->>API: POST /requirements
    API->>API: parse DTO
    API->>App: handleCreate
```

```
    App->>Domain: Requirement.create
    Domain->>Domain: validate fields
    Domain-->>App: Ok (Requirement + Event)
    App->>Infra: repo.Save
    Infra-->>App: Ok
    App->>Infra: publisher.Publish
    Infra-->>App: Done
    App-->>API: Ok (requirementId)
    API-->>Client: 201 Created
```

---

3. Frontend Architecture

3.1 Plugin System

```
graph TB
    subgraph "Plugin Layer"
        ReqPlugin["RequirementsPlugin"]
        FuturePlugin1["IdeationPlugin (planned)"]
        FuturePlugin2["ProjectManagementPlugin (planned)"]
    end

    subgraph "Core Infrastructure"
        PluginAbstract["SuitePlugin (abstract)"]
        Registry["PluginRegistry (singleton)"]
        ApiClient["ApiClient (Dio wrapper)"]
    end

    subgraph "Feature: Requirements"
        Domain["Domain - Model + Enums"]
        Data["Data - RequirementsRepository"]
        Presentation["Presentation - Notifier + Pages + Widgets"]
    end

    ReqPlugin --> PluginAbstract
    ReqPlugin --> Registry
    ReqPlugin --> Feature
    Feature --> Domain
    Feature --> Data
    Feature --> Presentation
    Data --> ApiClient
    FuturePlugin1 --> PluginAbstract
    FuturePlugin2 --> PluginAbstract
    Registry --> PluginAbstract
```

3.2 State Management (Riverpod)

```
graph LR
    subgraph "Riverpod Providers"
        RepoProv["requirementsRepositoryProvider"]
        NotifProv["requirementsNotifierProvider - keyed by projectId"]
    end

    subgraph "State (sealed class)"
        Init["RequirementsInitial"]
        Loading["RequirementsLoading"]
        Loaded["RequirementsLoaded"]
        Error["RequirementsError"]
    end

    subgraph "Widgets"
        Page["RequirementsPage"]
        Filter["StatusFilterBar"]
        Card["RequirementCard"]
    end

    NotifProv --> State
    Page --> NotifProv
    Page --> Filter
    Page --> Card
    RepoProv --> NotifProv
```

3.3 Navigation (go_router)

```
graph TB
    Router["GoRouter"]
    Shell["ShellRoute - _AppShell with NavigationRail"]
    ReqRoute["/projects/:projectId/requirements -> RequirementsPage"]

    Router --> Shell
    Shell --> ReqRoute
```

3.4 Frontend Layered Architecture

```
lib/
+-- core/
|   +-- api/api_client.dart               Dio wrapper, interceptors
|   +-- plugin_system/plugin.dart          SuitePlugin abstract class
|   +-- plugin_system/plugin_registry.dart singleton registry
+-- features/requirements/
|   +-- domain/requirement.dart            Model, enums, fromJson
|   +-- data/requirements_repository.dart  HTTP calls via Dio
```

```
|   +-- presentation/
|       +-- bloc/requirements_notifier.dart Riverpod StateNotifier
|       +-- pages/requirements_page.dart   Main screen
|       +-- widgets/
|           +-- requirement_card.dart      Card with status popup
|           +-- status_filter_bar.dart     Filter chips
+-- plugins/
|   +-- requirements_plugin.dart          Glue: plugin to feature
+-- main.dart                             Entry, Providers, Router, Shell
```

---

4. The Requirements Lifecycle Process

The core business process in Winslow is the lifecycle of a requirement — from initial capture through review, approval, and implementation.

4.1 High-Level Overview

```
stateDiagram
    [*] --> Draft: Create Requirement
    Draft --> UnderReview: Submit for Review
    Draft --> Rejected: Reject Immediately
    UnderReview --> Approved: Accept
    UnderReview --> Rejected: Decline
    UnderReview --> Draft: Request Changes
    Approved --> Implemented: Mark Done
    Implemented --> [*]
    Rejected --> Draft: Revise and Resubmit
```

The state machine is defined in `Winslow.Domain.Requirements.RequirementTypes` and enforced at the domain level:

From	Allowed Transitions	Guard
Draft	UnderReview, Rejected	-
UnderReview	Approved, Rejected, Draft	-
Approved	Implemented, UnderReview	-
Rejected	Draft	-
Implemented	-	Terminal state

Invalid transitions produce `DomainError.InvalidTransition("Draft", "Approved")`.

4.2 Detailed Walkthrough

4.2.1 Creation

1. **User** fills a form on the frontend with title, description, MoSCoW priority, kind, and acceptance criteria.
2. **Frontend** calls `POST /requirements` via `RequirementsRepository.create()` using the Dio HTTP client.
3. **API** receives `CreateRequirementDto`, parses string fields into domain types (`parsePriority`, `parseKind`), hardcodes the demo author ID, and dispatches `handleCreate`.
4. **Application** maps the command to `CreateRequirementInput` and calls domain `Requirement.create()`.
5. **Domain** validates:
   - Title is non-empty (wrapped in `NonEmptyString`)
   - Acceptance criteria list is not empty
   - All enum values are valid
6. On success, a `Requirement` aggregate is created in `Draft` status and a `RequirementCreated` event is emitted.
7. The requirement is persisted (in-memory or PostgreSQL) and the event is published (console-logged or queued).

4.2.2 Status Transitions

```
sequenceDiagram
    participant U as User
    participant FE as Flutter UI
    participant N as Notifier
    participant API as API
    participant App as Application
    participant Dom as Domain
    participant Infra as Infrastructure

    U->>FE: Change status
    FE->>N: transitionStatus
    N->>N: Optimistic local update
    N->>API: PATCH /requirements/{id}/status
    API->>App: handleTransition
    App->>Infra: repo.FindById
    Infra-->>App: Ok requirement
    App->>Dom: transitionStatus
    Dom->>Dom: validate state machine
    Dom-->>App: Ok (updated, event)
    App->>Infra: repo.Save
    App->>Infra: publisher.Publish
    Infra-->>App: Ok
    App-->>API: Ok
    API-->>N: 200 OK
    alt Failure
        N->>N: Revert optimistic update
```

```
        N->>FE: Show error
    end
```

Key design decisions in this flow:

- **Optimistic UI**: The frontend updates local state immediately, then reconciles on API response.
- **Domain enforcement**: The state machine guard is implemented in pure F# in the Domain layer. Even if the frontend allows an invalid transition, the backend rejects it with 400.
- **Event sourcing readiness**: Every state transition emits a `RequirementStatusChanged` event, enabling audit trails and event-driven integrations.

4.2.3 Status Display Rules

Status	UI Color	Description
Draft	Grey	Initial state, editable
UnderReview	Orange	Awaiting approval
Approved	Green	Ready for implementation
Rejected	Red	Declined, can be revised
Implemented	Blue	Done, terminal

---

5. Data Storage & Deployment

5.1 Storage Strategy

```
graph LR
    subgraph "Development (no Docker)"
        IM["InMemoryRequirementRepository - seed: 2 requirements"]
    end

    subgraph "Production (Docker Compose)"
        PG[("PostgreSQL 16 - volume: postgres_data")]
        MIG["migrations - auto-run on init"]
    end

    API["Winslow.API"] -->|DB_CONNECTION| PG
    API -.->|fallback| IM
```

Currently, the in-memory repository is always used — the PostgreSQL adapter is not yet implemented despite the Docker Compose and migration files being ready.

5.2 Docker Deployment

```
docker-compose.yml
+-- service: postgres
|   +-- image: postgres:16-alpine
|   +-- port: 5432
|   +-- volume: postgres_data (persistent)
|   +-- volume: ./backend/migrations -> /docker-entrypoint-initdb.d/
|
+-- service: api
    +-- build: ./backend/Dockerfile (multi-stage .NET 10)
    +-- port: 5000
    +-- env: DB_CONNECTION, ASPNETCORE_ENVIRONMENT, ASPNETCORE_URLS
    +-- depends_on: postgres
```

5.3 Database Schema

Three tables defined in `001_initial_schema.sql`:

- **`projects`** — UUID PK, name, description, status, methodology, owner, timestamps
- **`requirements`** — UUID PK, FK to projects, title, description, status, priority, kind, acceptance_criteria (JSONB), author, timestamps. Indexed on project_id, status, priority.
- **`domain_events`** — UUID PK, aggregate_id, event_type, payload (JSONB), occurred_at. Indexed on aggregate_id, event_type.

All UUIDs are generated via PostgreSQL's `pgcrypto` extension (`gen_random_uuid()`).

---

6. Railway-Oriented Programming Pattern

The entire backend uses a functional error-handling approach:

```
graph LR
    subgraph "Sync (Domain)"
        R1["Input"] -->|result builder| R2["Pure function"]
        R2 -->|Ok| R3["Success"]
        R2 -->|Error| R4["DomainError"]
    end

    subgraph "Async (Application)"
        A1["Command"] -->|taskResult builder| A2["repo.FindById"]
        A2 -->|Ok| A3["Domain.transitionStatus"]
        A3 -->|Ok| A4["repo.Save"]
        A4 -->|Ok| A5["publisher.Publish"]
        A5 -->|Ok| A6["Ok ()"]
```

```
        A2 -->|Error| A7["return Error"]
        A3 -->|Error| A7
        A4 -->|Error| A7
    end

    A2 -.-> R2
```

Custom builders (since F# 10 removed built-in `result { }`):

```
// Domain - sync
type ResultBuilder() =
    member _.Bind(m, f) = Result.bind f m
    member _.Return(x)  = Ok x
let result = ResultBuilder()

// Application - async + Result combined
type TaskResultBuilder() =
    member _.Bind(m, f) = task {
        let! r = m
        match r with
        | Ok v    -> return! f v
        | Error e -> return Error e
    }
let taskResult = TaskResultBuilder()
```

This allows mixing `Result`-returning domain functions and `Task<Result>`-returning repository calls in a single `taskResult { }` expression without manual nesting.

---

7. Error Handling & Status Code Mapping

```
graph TD
    API["API Handler"] -->|Result Ok| JSON["200 / 201 JSON"]
    API -->|Result Error| ErrorHandler["handleError"]
    ErrorHandler -->|Domain.NotFound| NotFound["404 Not Found"]
    ErrorHandler -->|Domain.ValidationError| BadReq["400 Bad Request"]
    ErrorHandler -->|Domain.InvalidTransition| BadReq
    ErrorHandler -->|Persistence| ServerErr["500 Internal Server Error"]
    ErrorHandler -->|External| ServerErr
```

---

8. Project Structure Summary

```
Winslow/
+-- backend/
```

```
|   +-- Winslow.sln
|   +-- docker-compose.yml
|   +-- Dockerfile
|   +-- migrations/001_initial_schema.sql
|   +-- src/
|   |   +-- Winslow.Domain/              Pure domain logic
|   |   +-- Winslow.Application/         Use cases
|   |   +-- Winslow.Infrastructure/      Adapters
|   |   +-- Winslow.API/                Falco HTTP
|   +-- tests/                           Empty placeholder
|
+-- frontend/
    +-- pubspec.yaml
    +-- lib/
    |   +-- main.dart                    Entry point
    |   +-- core/
    |   |   +-- api/api_client.dart      Dio wrapper
    |   |   +-- plugin_system/           Plugin system
    |   +-- features/requirements/       Feature module
    |   +-- plugins/                     Plugin registration
    +-- test/
```

---

9. Key Architectural Decisions

Decision	Rationale
4-layer DDD backend	Separation of concerns; Domain has zero deps, Application orchestrates, Infrastructure adapts, API serves
Railway-Oriented Programming	No exceptions for control flow; explicit `Result<Ok, Error>` forces handling at every level
Custom CE builders	F# 10 removed built-in `result { }`; custom builders provide same ergonomics
In-memory repository default	Zero-setup development; PostgreSQL adapter ready for production swap
Plugin-based frontend	Feature isolation - each domain is a self-contained plugin
Riverpod with override pattern	Clean dependency injection - repository provider throws by default, overridden in `main.dart`
Optimistic UI updates	Instant feedback for status transitions; reconciled on API response

Decision	Rationale
Manual serialization (no freezed)	Simplicity for current scale; toolchain is ready for future code-gen adoption